RESEARCH AND IMPLEMENTATION PROPOSAL

Simplifying the K215 correctness proofs

Semantic interfaces, smaller imports, and shared logical derivations

Prepared for Vladimir Reshetnikov
ProveIt / Busy Beaver / ZFC

21 October 2026

Main recommendation. First remove an accidental import of the older 254-state program proof from the K215 semantic calibration. Then make the register-machine simulation consume semantic block contracts, so a shared decoder proof and primitive proofs can replace repeated whole-site execution. A third prototype shares the named-calculus reasoning while retaining the essential restriction on equality atoms.

Measured immediate reduction. The import patch changes the selected K215 source closure from 192 to 181 modules, removes a net 198,510 source bytes and 3,133 source lines, and keeps the old public declarations available. These are import-closure measurements, not proof-term or timing measurements.

Scope. This packet supplies reviewable patches, mathematical arguments, targeted Lean checks, and reproducible source measurements. It proposes no machine-state reduction and makes no new unconditional nonhalting claim. The canonical 215-state transition table is untouched.

Pinned repository snapshot

`2ca3a5c16f0876d2cab842056353c03a1ed2f5f0`

github.com/VladimirReshetnikov/ProveIt

Contents

1 What should be simplified next? 1

2 The theorem and the existing proof architecture 1

3 Immediate reduction: separate ZF semantics from the old program 3

4 The main architectural proposal: semantic site contracts 5

5 What can be shared, and what cannot yet be discarded 7

6 Sharing logical proofs without strengthening the calculus 9

7 Applying the same approach to Rocq 11

8 Validation, trust, and reproducibility 12

9 Recommended implementation sequence 13

A Packet contents and use 14

B Source guide 14

C Attribution 15

1 What should be simplified next?

The best next simplification is an architectural one: let each proof layer state the mathematical behavior it needs, and keep the particular checker that establishes that behavior behind an adapter. The present repository has already made substantial progress in this direction. The remaining opportunity is to complete the separation at two precise boundaries: the import boundary between set-theoretic semantics and the older 254-state program, and the interface between site validation and the machine simulation.

This report studies the source at commit `2ca3a5c16f`, rather than treating all earlier research packets as current production code. The latest Lean implication core has already been reduced to a 58-step condensed-detachment derivation; shared bundle semantics, normalized source correspondence, shared Home return, and exact decoder-prefix proofs already exist. Recommending these again would not answer the next-step question. The repository also contains older verification receipts and unadopted proposals; their status must be distinguished from the bytes currently imported by an endpoint.

Priority	Concrete change	What it achieves
1. Smaller imports	Extract `ZFNative.models_ZF`; bypass the unused `WFTheory` import	192 → 181 modules in K215's import closure; old theorem names and legacy modules retained.
2. Semantic sites	Add `PortContract`, `EdgeContract`, and `SemanticCovered`; make the simulation use them	Allows direct composition of decoder and primitive proofs without re-proving acceptance by the bounded symbolic executor.
3. Shared logic	Factor classical and quantifier reasoning through `NamedDerived`, with explicit equality-atom admissibility	Removes duplicated proof bodies while preserving the distinction between the unrestricted and equality-free calculi.
4. Rocq alignment	Reconcile the retained Rocq source lane with the normalized/decode-based checked successor; port the semantic interface	Uses already available checked work and applies the same architectural separation to Rocq's stream semantics.

Table 1: The proposed order of work. A smaller import closure is an immediate structural reduction; the semantic interface is the larger subsequent proof-sharing opportunity.

The second and third changes initially add infrastructure. Their value must not be reported as a line-count saving before duplicated clients are removed. In particular, the primitive contracts needed for complete replacement of `KSites215` remain explicit obligations. The packet's verification summary states exactly which prototypes were compiled and which endpoint builds were not performed.

2 The theorem and the existing proof architecture

2.1 The exact endpoint

The Lean endpoint in `KEndpoint215.lean` is

```
BlankHalts machine215 ⟷ BProv ZFCax_s [] Form.fBot,
¬BlankHalts machine215 ⟷ ¬BProv ZFCax_s [] Form.fBot.
```

Thus the right side is the absence of a finite derivation of falsity from the repository's explicit ZFC axiom predicate. This is the syntactic consistency statement used by the endpoint. The nonhalting

equivalence is obtained from the halting/refutation equivalence by `not_congr`; it should remain that short.

The formal theorem is expressed in Lean's metatheory. The corresponding Rocq endpoint is expressed in Rocq's metatheory. These facts are distinct from a separately represented object-language derivation of the equivalence inside ZFC or PA. The proposals here preserve the existing theorem and its intended formalization in ZFC; they do not create a new internal arithmetic endpoint. No step assumes Con(ZFC), and no step proves that the machine actually runs forever without a consistency premise.

The existing Lean endpoint composes the following independent mathematical claims. The separation is already good and should be retained.

1. **Dense/sparse correspondence.** The literal dense machine with 215 working states is a relabeling of the maintained sparse table. The halting state and blank initial configuration are handled explicitly.
2. **Bootstrap and tape representation.** The blank machine reaches the dispatch representation of the empty source path and the chosen register seed, including the next-candidate value 4.
3. **Physical simulation.** Every source command is implemented by one or more concrete Turing-machine transitions, with exact path, gap, and register behavior.
4. **Source/enumerator correspondence.** The optimized source and the command-permuted abstract proof enumerator have equivalent halting behavior.
5. **Proof-search calibration.** The enumerator halts precisely when the falsity code is derivable in the compact calculus.
6. **Logical calibration.** Derivability of that code is equivalent to a ZF refutation, and the existing constructibility argument transfers refutability between ZF and ZFC.

The endpoint composition is visible in `KEndpoint.lean` and `KEndpoint215.lean`. The other main interfaces are in `EnumPerm.lean` and `ConstructibleChoice.lean`. Keeping logical calibration independent of machine engineering is particularly valuable: future 215-state variants should reuse it without repeating set theory.

2.2 What is already present

The phase-nine report records a 192-module endpoint closure and a 197-module selected union that also includes five decoder-prefix modules. It records 4,751 site leaves and 6,176 branch outcomes, with the shared Home return already used by all nine site batches. It reports 984.608 seconds of fresh site compilation, versus 1,649.340 seconds in the older phase-six records, explicitly as a historical comparison rather than a paired benchmark. [Phase-nine record]

The latest `DominatedImplication.lean` has 295 lines and its `DomRho8.lean` compatibility adapter has 73. The 58-step derivation was compiled locally in the repository's recorded follow-up; it is later than the complete phase-nine endpoint check. [Condensed-detachment status]

The shared semantics in `KSeed222.lean` and `KSeedModel222.lean` already avoid repeating the bundle interpretation in two directions. The generic decoder entry in `KDecoderPrefix215.lean` retains the entire consumed source word and locates a primitive port uniformly over every gap and register list. These are foundations for the next change, not proposed discoveries of this packet.

2.3 Current-source and receipt comparison

The source audit in this packet compares hashes rather than inferring current status from a README. Its results are summarized below. All comparisons are at the pinned commit, not against a later moving branch.

Receipt compared with current source	Match	Differ	Missing
Lean phase-nine selected union	196	1	0
Rocq original local 42-file lane	41	1	0
Rocq normalized 44-file successor	34	6	4

The Lean difference is `DominatedImplication`, replaced by the later 58-step core. The current Rocq tree largely matches the earlier 42-file local lane; its differing `DomRho8` matches the compact replay in the normalized receipt. The six differences from the normalized successor are `EnumPerm`, `L1_SoundPerm`, `L1_CompPerm`, `SeedPerm`, `L2RootPerm`, and `Source215`. The four normalized modules absent from the current generated lane are `PathCodec`, `BisimMirror`, `NormalizedBisim`, and `NormalizedCert215`.

This identifies a retained older Rocq implementation and stale applicability of some descriptions; it is not evidence that the mathematical equivalence is false. It does mean that a receipt for one exact source set must not be used as an exact-source verification claim for another. The reproducible comparison is included in `audit_snapshot.py` and `snapshot_audit.json`.

3 Immediate reduction: separate ZF semantics from the old program

3.1 The unnecessary dependency

The current `ZFNative.lean` combines several kinds of result. One of its theorems, `ZFNative.models_ZF`, extracts semantic fields of a ZF model from satisfaction of the explicit ZF axiom predicate. That theorem does not refer to an enumerator or a register program. The same file also contains corollaries about the older program and imports `NativeTheory254.lean`, which imports `L2Root254.lean`. Consequently a consumer that needs only the semantic extraction theorem imports the older program proof too.

There are two routes into the K215 endpoint:

- `ConstructibleChoice.lean` imports `ZFNative` for its semantic ZF-model theorem.
- `KQ1ZF.lean` imports `WFTheory`, although the relevant development needs the model interface already supplied by `WFModel`. The stronger `WFTheory` imports `ZFNative` in turn.

This is a dependency problem rather than a hard logical proof. The right solution is to expose the small semantic interface directly.

3.2 The four-file patch

The supplied `semantic_import_cut.patch` does exactly the following.

1. Introduce `ZFModelSemantics.lean`, importing only `InternalOrd` and containing the existing theorem `ZFNative.models_ZF` under its original namespace.

2. Make `ZFNative.lean` import the new module, and remove the duplicate original declaration from that file. All remaining legacy declarations stay available.
3. Change `ConstructibleChoice`'s import from `ZFNative` to `ZFModelSemantics`.
4. Change `KQ1ZF`'s import from `WFTheory` to `WFModel`.

The moved theorem body is byte-for-byte identical. Its statement is

$$(\forall f,\ \mathrm{ZFax_s}(f) \to \mathrm{Sat}(\mathrm{mem}, v, f)) \\ \Longrightarrow \mathrm{ZFAxioms}(\mathrm{mem}) \ \wedge\ (\forall a\, \exists p\, \forall x, x\ \mathrm{mem}\ p \leftrightarrow \mathrm{Sub}(\mathrm{mem}, x, a)) \ \wedge\ \mathrm{Regular}(\mathrm{mem}).$$

The proof simply extracts the finitely named axiom forms and the two schemas, then applies the existing interpretation lemmas. Moving it does not replace that argument with an assumption or a new axiom.

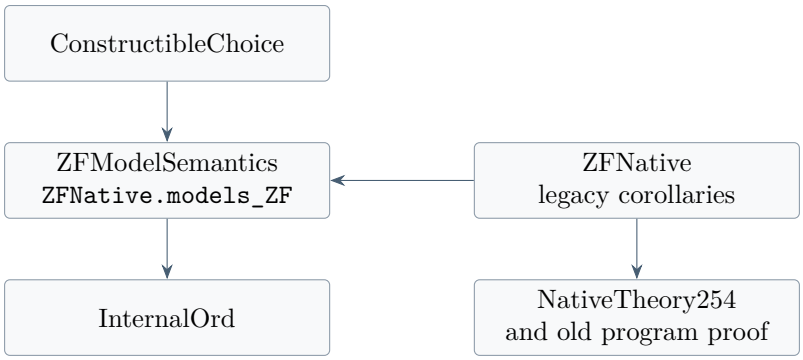


Figure 1: The proposed import direction. The semantic consumer no longer imports the old program; existing clients of `ZFNative` still do.

3.3 Measured effect and its limits

The source closure of `BusyBeaver.ZFC.KEndpoint215` changes as follows:

Quantity	Before	After	Net reduction
Project modules	192	181	11
Source bytes	2,363,203	2,164,693	198,510
Source lines	45,143	42,010	3,133

Twelve old modules leave the endpoint's import closure and one new module enters. The old modules include `Dag254`, `DecodeFresh254`, `L2Lib254`, `L2Macros254`, `L2Control254`, `L2Axioms254`, `L2AxiomsEval254`, `L2Root254`, `NativeDecoded254`, `NativeTheory254`, `WFTheory`, and `ZFNative`. They remain in the repository for legacy clients.

The resulting reductions are approximately 5.73% of imported modules, 8.40% of imported source bytes, and 6.94% of imported source lines. The metrics include comments and source data; they are not a theorem-term dependency analysis. An import can force a module to be compiled even if a final theorem's proof term never refers to most declarations in that module. Accordingly this patch simplifies the build and review closure without claiming a corresponding percentage reduction in logical assumptions, proof-object size, or running time.

The patch is deliberately independent of the machine table, source certificate, site execution checks, and logical statement. It can be reviewed and adopted before the larger simulation refactor.

4 The main architectural proposal: semantic site contracts

4.1 Why the current interface blocks reuse

In KSim.lean, the current coverage predicate requires, for every input path, a decoded leaf whose `checkEdge` evaluates to `true`. That checker uses a particular symbolic executor, a fixed fuel of 4,000, and a particular symbolic target representation. Its soundness theorem then supplies the concrete execution facts needed by the simulation.

A direct proof of a primitive's behavior does not imply that this specific bounded executor accepts its site. Establishing such acceptance would need a separate completeness or boundedness argument about the executor, even when the desired concrete Turing-machine run has already been proved. That extra obligation serves the implementation of the checker, not the final correctness theorem.

The proposed change makes the needed concrete execution facts the public interface. Existing Boolean certificates remain usable through a soundness adapter. Shared proofs can supply the same interface directly. There is no assumption that the checker recognizes every correct block.

Earlier decoder-sharing proposals already identified the need for such an interface. The contribution here is its checked implementation, compatibility clients, exact connection to the maintained decoder prefix, and a checked guarded primitive pilot.

4.2 Precise configurations

Let Q be the full consumed source word, $n \in \mathbb{N}$ the variable gap parameter, and $\vec{v}$ the physical register list. The current symbolic dispatch configuration is `start Q`; its concrete realization is

$$I(Q, n, \vec{v}) = \mathrm{zc}\Big(84, [1], Q\,0^{\,n+(64-|Q|)}\,1\ \mathrm{sepsR}(\vec{v})\Big).$$

Here the list operations and natural-number subtraction are exactly those of `conc_start`; the coverage contract requires $|Q| \le 48$. The symbolic target `tgtConf` records the next path, overflow bit, and register-box effect. Existing lemmas already show how its concrete realization reconstructs the next representation relation.

For a successful source transition, write $x \longrightarrow_M^+ y$ for *positive* finite reachability in the physical machine, and $x \longrightarrow_M^* y$ for possibly zero transitions. Both conventions are explicit in the code as `Reach1` and `Reach`.

Definition 4.1 (Run and port contracts)**.** A `RunContract M br source target` is the proposition

$$\forall n\,\vec{v},\ \mathrm{Valid}(\mathrm{br}, \vec{v}) \longrightarrow \mathrm{conc}(n, \vec{v}, \mathrm{source}) \longrightarrow_M^+ \mathrm{conc}(n, \vec{v}, \mathrm{target}).$$

A `PortContract M source Q e` pairs the length and register-index bounds with the appropriate run contracts for the edge e. A halt edge instead requires finite reachability to state 230 for every sufficiently long register list; an internal node edge has contract $\bot$. The dispatch specialization is

$$\mathrm{EdgeContract}_M(Q, e) := \mathrm{PortContract}_M(\texttt{start Q}, Q, e).$$

For a decrement edge the port contract contains *two* branch-sensitive proofs. The zero-register branch returns to `breakPath 1 Q` without decrementing; the nonzero branch returns to `incPath Q` and decrements the selected register. A Boolean or propositional test on a concrete small sample of registers is insufficient: these contracts quantify over arbitrary register values and gaps.

4.3 Coverage by semantics

Definition 4.2 (Semantic coverage)**.** For a register program p, SemanticCovered(M, p) asserts

$$\forall P\ \exists e, d, \quad \begin{array}{l} \mathrm{walk}(p, p.\mathrm{nnodes} + 1, p.\mathrm{root}, P, 0) = \mathrm{some}(e, d), \\ \mathrm{EdgeContract}_M(\mathrm{consume}(P, d), e). \end{array}$$

The consumed word includes the walk's implicit zero padding.

The patch proves `Covered.toSemantic`: the current Boolean coverage and the usual register-box lemmas imply semantic coverage. It also proves `SemanticCovered.of_leaves`: total source shape plus a semantic contract for every literal leaf imply semantic coverage. The source-shape check can be performed independently of symbolic Turing-machine execution.

The existing `sim_step`, `sim_halt`, and `halts_iff` signatures remain available as compatibility adapters. There is one shared semantic simulation proof. A small legacy `sim_stop` wrapper retains its existing weaker parameter list; it needs no register-box lemmas.

Theorem 4.3 (Simulation from semantic coverage)**.** *Assume* SemanticCovered(M, p)*, both actions of terminal state 230 are absent, and the source configuration c and physical configuration T satisfy the existing representation relation* $\mathrm{Rel}(p, c, T)$*. Then*

$$\mathrm{Halts}_p(c) \quad \longleftrightarrow \quad \mathrm{Halts}_M(T).$$

Proof. The source walk identifies a leaf and its exact consumed word. The clean-path invariant rewrites the physical tape into that leaf's dispatch representation. The semantic contract for the decoded edge supplies positive physical reachability to the next representation, including the same register update, or supplies reachability to the terminal state for a halt edge.

For the forward direction, induct on the number of source transitions to the halt leaf and compose the concrete runs. For the reverse direction, induct strongly on the length t of a physical halting run. If the source has already stopped, coverage excludes a failed decode or internal node, so it is at its halt leaf. Otherwise the semantic contract produces a positive block of length $j + 1$. Determinism implies $j + 1 \le t$, since a successful run cannot continue through a halted configuration. The remaining physical halting run has length $t - (j + 1) < t$, so the induction hypothesis applies. □

This is the mathematical argument already present in `KSim`; the patch changes its input interface rather than introducing a new simulation principle. No source/physical clock equality is required.

4.4 Why positive progress must remain

Replacing $\longrightarrow^+$ by $\longrightarrow^*$ in the nonhalting edge contracts would invalidate the reverse direction. For example, relate a source state that loops to a physical state that has already halted. Every source step could then be matched by zero physical steps, while the physical machine halts and the source does not. The positive-step obligation excludes this example and makes the induction on the remaining physical time well founded.

A decoder prefix may legitimately have length zero: a primitive could begin at the initial control position. The *whole successful macro step*, including the primitive and return, must have positive length. This is exactly the distinction implemented by `PortContract.precompose`.

Lemma 4.4 (Prefix composition)**.** *Suppose, uniformly in every $n, \vec{v}$, the dispatch configuration reaches a primitive port in zero or more steps. If that port satisfies* $\mathrm{PortContract}_M(\mathrm{port}, Q, e)$*, then* $\mathrm{EdgeContract}_M(Q, e)$.

Proof. For each nonhalting branch, compose the prefix's reflexive-transitive run with the positive suffix run. For a halt edge, compose the prefix with reachability to state 230. Bounds on the full word and register index are inherited from the port contract. This proof treats every branch separately, retaining its register-validity hypothesis. □

4.5 Connection to the existing exact decoder prefix

The current `KDecoderPrefix215.leaf_prefix` already gives a uniform primitive entry. It chooses a decomposition $Q = \mathrm{pre}\,[b]$, a physical state q, and a routed source role before quantifying over the gap and register list. Its target zipper retains reverse(pre)[1] on the left and the unread bit, gap, marker, and registers on the right.

The new `KSemanticSites215` module composes that theorem with an explicit `PrimitiveContracts` premise. It provides the implication

$$\mathrm{PrimitiveContracts} \implies \mathrm{SemanticCovered}(\texttt{tm215}, \texttt{p215}).$$

The premise is visible in every resulting theorem; it is not an axiom and is not claimed to have been established for all primitive families in this packet. The resulting source/machine equivalence is therefore an integration theorem, not a replacement completed proof of the full 215-state endpoint.

5 What can be shared, and what cannot yet be discarded

5.1 A reproducible census of primitive obligations

The exact `K215Decoder` source/role arrays give the following census. The source has 202 nodes and the decoder certificate 234 roles. Its longest consumed word has 47 bits. The script in the packet parses the literal arrays, walks the source DAG, counts primitive role entries, and records the pinned input hash.

Operation	Occurrences	Outcomes	Role entries	Role outcomes
Increment	1,900	1,900	32	32
Decrement or zero	1,425	2,850	38	76
Break	98	98	7	7
Continue	1,327	1,327	19	19
Halt	1	1	1	1
Total	4,751	6,176	97	135

The ratio $6176/135 \approx 45.75$ measures potential sharing of role-outcome families. It is *not* a measured speedup, a proved bound of exactly 135 lemmas, or evidence that all occurrences with a common physical state have the same tape semantics. Some families may need additional guarded subcases.

5.2 The continuation is part of the state

Two source leaves can reach the same physical state and read the same bit yet have different words to the left of the head. A break or continue instruction can inspect that prefix, so those entries cannot be identified just by a physical state number. Similarly, a role's short suffix is insufficient to recover the full source continuation.

The right grouping key is a *parametric theorem*: the routed role, operation and branch condition together with a quantified full prefix satisfying an explicit invariant. The source-to-role certificate establishes that every occurrence falls under one such theorem. The theorem then implements the operation for all admissible prefixes, natural gaps and register values.

This is why the proposed interface retains Q both in the port contract and in the target path. The exact source walk, its consumed length, and the primitive's update must all remain connected in the final proof.

5.3 An incremental replacement strategy

A practical implementation can maintain a mixture of proof producers. For each leaf, either an existing Boolean check supplies its semantic contract or a new family lemma supplies it. There is no need to replace all 4,751 leaves at once. This supports the following sequence:

1. Introduce the semantic interface and retain all current site checks.
2. Establish one guarded family at an exact primitive entry; retain its full prefix and prove the register and gap behavior for all natural values.
3. Prove that every intended occurrence satisfies that family's guard, using the source/role certificate. Remove only those corresponding old checks.
4. Repeat for decrement branches, break, continue, and the halt case.
5. Once every leaf has a semantic provider, remove the remaining repeated whole-site execution data and retain the small source-shape check.

The phase-nine packet contains a source-only first CB1 suffix proposal, associated with 86 literal occurrences and an explicit length guard. The current packet implements and compiles a successor of that pilot, keeping the global occurrence-coverage obligation separate. Compiling a suffix theorem by itself does not prove that all 86 proposed sites satisfy its entry predicate.

5.4 Checked pilot: the CB1 continuation

The included `KCB1Suffix` and `KCB1Contract` modules prove one concrete family at the semantic interface. Write the consumed word as $Q = A001$. At the primitive port the machine is in state 108 reading the last 1. The three maintained transitions are

$$(108, 1) \mapsto (0, L, 10), \qquad (10, 0) \mapsto (0, L, 16), \qquad (16, 0) \mapsto (1, L, 1).$$

They rewrite $A001R$ to $A100R$, with the head ready to scan the unchanged prefix A. A finite traversal guard certifies that state 1 returns over A and the left marker to state 2; the established Home boundary then returns to the dispatch state 84.

The explicit guard is

```
leftCheck tm215 1 (A.reverse ++ [true]) 2 = true
```

and the length premise is $|A| + 3 \leq 48$. The unchanged-word traversal checker has its own soundness proof. The frame theorem is uniform in an arbitrary right tape frame R, so no register bound or numerical register sampling is hidden in this pilot.

The source-path identities are proved separately:

$$\mathrm{stripPath}(0, 1, A001) = A0, \qquad \mathrm{breakPath}(1, A001) = A1,$$

and the overflow flag is false. The two trailing zero cells become part of the gap; under the length premise,

$$n + (64 - (|A| + 3)) + 2 = n + (64 - (|A| + 1)).$$

This equality is important because subtraction on natural numbers saturates. The accepted proof makes the source and target zipper equalities explicit; the preserved draft's list-association and gap-normalization issues are repaired.

`KCB1Contract.port_contract` proves the all-gap, all-register primitive contract for the actual maintained table. `edge_contract` composes it with `leaf_prefix`, assuming the exact source-leaf and routing facts to state 108 reading true. The remaining task before retiring the 86 proposed checks is to certify those routing facts and traversal guards for every intended occurrence. This packet does not claim that finite coverage theorem.

6 Sharing logical proofs without strengthening the calculus

6.1 The repeated mathematical argument

`TarskiDerived.lean` and `KThmK.lean` contain parallel named-formula reasoning: deduction from hypotheses, syllogism, contraposition, double negation, implication case analysis, quantifier monotonicity, and vacuous quantification. The two derivability predicates are not identical. In particular, the restricted calculus must not acquire atomic equality-vacuity instances just because the unrestricted calculus has them.

A previous structural proposal suggested parameterizing this repeated development. The `NamedDerived` prototype makes that suggestion concrete while exposing the exact restriction required by the second client.

6.2 The common interface

Let NF be the existing named-formula type and let $D : \mathsf{NF} \to \mathsf{Prop}$ be an arbitrary theorem predicate. The generic module takes:

- the existing `Kalmar.Classical` interface for implication, falsity, modus ponens, and the classical propositional basis;
- four quantifier operations: generalization, distribution of universal quantification over implication, commutation of quantifiers, and the relevant $\exists i\, \forall i$ elimination principle;
- explicit atomic-vacuity hypotheses, including a predicate specifying which equality atoms admit them.

No numerical formula coding, Cantor pairing, TM state, source program, or decoder certificate occurs in this interface.

Definition 6.1 (Admissible formulas)**.** For $E : \mathbb{N} \to \mathbb{N} \to \mathsf{Prop}$, define recursively

$$\begin{aligned} \mathrm{Adm}_E(\mathrm{rel}(a,b)) &= \top, \\ \mathrm{Adm}_E(\mathrm{eq}(a,b)) &= E(a,b), \\ \mathrm{Adm}_E(\bot) &= \top, \\ \mathrm{Adm}_E(A \to B) &= \mathrm{Adm}_E(A) \wedge \mathrm{Adm}_E(B), \\ \mathrm{Adm}_E(\forall j\, A) &= \mathrm{Adm}_E(A). \end{aligned}$$

Choosing $E \equiv \top$ permits all formula atoms. Choosing $E \equiv \bot$ restricts to equality-free formulas. The generic theorem then has the same induction for both clients, while its premises prevent the restricted client from being silently strengthened.

Theorem 6.2 (Guarded vacuous quantification)**.** *Under the common classical and quantifier interfaces and atomic-vacuity hypotheses, if* $\mathrm{Adm}_E(A)$ *and the repository's syntactic predicate* $\mathrm{Occurs}(i, A)$ *is false, then*

$$D(A \to \forall i\, A).$$

Proof. Induct on A. Relation and equality atoms use their respective atomic hypotheses; the equality case requires the admissibility witness $E(a, b)$. Falsity follows from ex falso.

For $A = B \to C$, the induction hypotheses give vacuous quantification of B and C. The common quantifier rules derive vacuous quantification of $\neg B$. Universal monotonicity then gives $\neg B \to \forall i(B \to C)$ and $C \to \forall i(B \to C)$. The shared classical implication case lemma combines these into $(B \to C) \to \forall i(B \to C)$.

For $A = \forall j\, B$, apply universal monotonicity at j to the induction hypothesis and then commute the quantifiers. Each use of nonoccurrence is the same syntactic premise as in the original theorem, not an informal substitution of "not free" for a stronger existing condition. □

6.3 Compatibility and honest size accounting

The original theorem namespaces and statement shapes are retained as adapters. The packet includes a client that instantiates the original interfaces of both calculi and prints axiom reports for the generic and specialized vacuity theorems. A downstream `KEind222` check exercises the restricted-calculus consumer, rather than only compiling a standalone abstract lemma.

The first explicit-signature implementation grows the combined source rather than shrinking it. This is expected when two clients retain named public wrappers and a third file adds the common interface. The appropriate claim is one proof of the repeated reasoning and two checked specializations. The final compact-alias implementation has the following accounting:

Production file	Original lines	Proposed lines
`TarskiDerived`	183	136
`KThmK`	320	295
`NamedDerived`	0	168
Combined	503	599

The byte count grows from 21,694 to 23,660. Thirty-eight public derived-rule bodies become type-compatible transparent aliases; the two vacuity declarations remain theorems. This is a checked maintenance prototype, with an initial cost of 96 lines and 1,966 bytes, rather than the report's headline reduction. If physical source length is the overriding objective, adopt the import cut first and defer this factorization; flattening carefully named lemmas merely to save wrapper lines can make proof terms and maintenance worse.

7 Applying the same approach to Rocq

7.1 Use the actual retained source lane

The current Rocq lane has not adopted every change described in the later normalized receipt. The shortest immediate route is to reconcile and replay that existing checked successor, rather than independently rediscover the same normalization and command-level decomposition. Both maintained templates and generated files must be updated together. An edit only to `coq/215/Source215.v` is not a durable refactor if regeneration restores the older template.

The relevant existing improvements are a normalized source certificate with one correspondence and symmetry, command decoding followed by a small `xstep`, and a completeness interface valid from an arbitrary candidate boundary. The older seed file visibly duplicates a reachability induction; the normalized/decode-based successor already removes that duplication. The repository's earlier one-rank scheduler proposal can be evaluated separately if arbitrary initial proof-list pointers are desired. It is not new work in this packet.

7.2 The semantic boundary in the stream-based simulator

The concrete port is visible in `Sim230.v`. Its `inv_view` returns a successful `check_edge` equation; `inv_step` unpacks that Boolean result and calls `site_run'`, while `inv_halt` uses `site_halt'`. The proof then consumes only positive reachability and terminal behavior.

The Rocq adaptation should therefore introduce a proposition `edge_spec` whose cases state the concrete branch-sensitive runs, length and register-index bounds, and terminal reachability. Prove

$$\texttt{check_edge Q e = true} \implies \texttt{edge_spec Q e}$$

from the existing checker-soundness theorems. Have `inv_view` expose `edge_spec` and have `inv_step`/`inv_halt` consume it directly. The existing proof of `tm230_rm` in `Sim230Main.v` continues to use the same progress argument.

This port should use Rocq's own tape and progress relations. Replacing its stream semantics by Lean's zipper representation would create a second unnecessary correspondence theorem. Share the mathematical contract and the finite source/role data; let each prover retain its native representation and kernel-checked implementation.

7.3 What should be shared across provers

The most stable common artifacts are small, representation-neutral data: source/role graphs, normalized correspondence pairs, primitive operation tags, register permutations, and condensed-detachment dependency edges. Python can generate these aggressively because its output is checked by each kernel. The generator must not become an axiom about correctness.

The semantic proof should be written once per prover over an abstract interface. This is a much smaller duplication than re-proving each machine variant and each source occurrence. It also preserves useful independence: Rocq and Lean need not share tactic scripts or trusted evaluation mechanisms.

The current packet does not claim a new Rocq kernel run for the proposed semantic-site refactor. Its Rocq section is a precise port plan grounded in the actual current `Sim230` proof, while the implementation pilot and new compilation receipts are in Lean.

8 Validation, trust, and reproducibility

8.1 What the targeted checks establish

The packet uses Lean 4.32.0, the toolchain named by the repository. Each selected module is compiled into a fresh task-specific object directory, using the pinned source plus an explicit overlay. Unchanged objects can be reused only when the compiler identity, module bytes, direct imports, transitive source fingerprints, and output hashes agree.

This last requirement is material. A changed executable definition may affect kernel reduction in downstream `decide` proofs even if an intermediate compiled object happens to be byte-identical. The phase-nine history already records this failure mode. The supplied targeted driver keys reuse by the entire transitive source/import fingerprint, not just immediate object hashes.

The compact verification summary accompanies the raw per-module logs and source-hash ledger. A successful targeted build checks the included new theorem bodies and named compatibility clients. It is not a second full replay of the 215-state endpoint or the entire Rocq closure.

Accepted check	Modules	Checked scope
Import cut	123	99 freshly compiled, 24 matched providers reused; generic endpoint, semantic ancestors, original legacy facade and eight named assumption reports.
Semantic sites	27	Final source closure of `KSimCompatibility`, `KSemanticSites215`, and `KCB1Contract`; accepted providers and their original logs retained.
Shared named logic	16	13 freshly compiled, 3 matched providers reused; both old interfaces and the actual `KEind222` consumer.

The import-cut audit has no warnings; its inherited source ancestry retains twelve linter warnings in seven unchanged modules. The accepted new machine clients have no warnings. The complete machine-specific `KEndpoint215` closure was not rebuilt in this packet, and no new Rocq compilation is claimed. The three proposal checks are separate checks of their respective overlays; they do not assert that a combined production checkpoint has been replayed.

The `KSim` interface change itself grows that module from 356 to 506 lines. Its five supplied source modules, including audit and pilot clients, total 778 lines. Fifteen original executable/interface definitions are byte-identical, including `checkEdge`, `start`, `tgtConf`, `Covered`, and `Rel`. The subsequent removal of repeated leaf checks is where a net machine-layer source reduction can be measured.

8.2 Acceptance checks for each change

For the import cut, the essential checks are: the moved theorem has its original type and body; old `ZFNative` clients still compile; the changed semantic consumers and generic endpoint compile; and the

static import graph actually loses the twelve unwanted modules. There is no reason to recompute every physical site merely to validate those import edits.

For the semantic interface, check the old public `KSim` theorem types, the new semantic equivalence, the Boolean adapter, and the decoder-prefix composition. The first primitive-family pilot must quantify over arbitrary registers and gaps and retain every guard. A later replacement of actual site providers does require checking all changed source-dependent descendants and the concrete endpoint client.

For shared logic, check both specializations, a downstream restricted-calculus consumer, and the axiom reports. The type of the generic theorem must include the equality-atom admissibility premise; a proof of a stronger unrestricted calculus would not be an acceptable substitute.

8.3 Axiom reports and limitations

The repository's recorded closed Lean endpoint audits use `propext`, `Classical.choice`, and `Quot.sound`. The targeted prototype reports are listed in the delivered verification summary. An explicit theorem argument such as `PrimitiveContracts` is a hypothesis, not a newly asserted global axiom; it must still be discharged before the semantic route becomes a closed machine endpoint.

The moved `models_ZF` theorem and new `Semantic.halts_iff` use `propext` and `Quot.sound`. The source-shape check has no axioms. The CB1 port contract uses `propext` and `Quot.sound`; its composed edge contract additionally uses `Classical.choice` through the existing decoder prefix. The generic and specialized vacuity theorems report only `propext`. The original restricted specialization and alpha-renaming theorems retain their existing three standard axioms. No `sorry` or new postulate is introduced by the accepted prototypes.

No timing improvement is inferred from source measurements. No lower bound on machine size is inferred from a failed simplification. No claim that 135 role outcomes have all been proved follows from counting them. These distinctions make the packet usable as a concrete next development step rather than an overstated new correctness certificate.

9 Recommended implementation sequence

1. **Adopt the semantic import cut.** It is the smallest patch, moves one existing theorem verbatim, and immediately removes the obsolete program layer from K215's imported source closure. Keep legacy facades and public names.
2. **Adopt the semantic `KSim` interface with old checks intact.** This creates an independently checkable compatibility checkpoint. The machine table and current leaf certificates need no alteration at this stage.
3. **Complete one exact primitive family.** Use the established decoder port, the existing register-box contracts, and the shared Home return. Prove the full continuation law and then certify which source leaves instantiate it. Replace only those leaf checks.
4. **Scale by operation families.** Decrement has the largest outcome count and requires separate zero/nonzero branches. Break and continue need especially careful full-prefix invariants. Halt has its own terminal contract.
5. **Factor logical derivations if maintenance is the objective.** The generic `NamedDerived` proof removes duplicated reasoning but its compatibility wrappers initially increase source length. Keep that tradeoff explicit in review.

6. **Bring the Rocq lane to the chosen checkpoint.** Reuse the already checked normalized successor where appropriate, synchronize generators, and apply the same semantic interface over Rocq's existing relations. Then perform the maintained recursive kernel audit for the selected final source set.

The most valuable long-term shape is a short composition theorem whose inputs are independently named semantic contracts. The expensive work then lives in small reusable lemmas and finite certificates, with clear ownership of the source word, continuation, register effect, and positive physical progress.

A Packet contents and use

The archive contains the LaTeX source and compiled PDF of this report, separate overlays and patches for the proposed changes, audit clients, reproduction scripts, source measurements, and compilation receipts. It omits compiler binaries and compiled proof objects. `README.md` gives the exact layout and the final verification scope.

To review a patch against the pinned checkout, use `git apply -check` first. The overlays are alternatives to applying patches: the supplied `scripts/check_lean.py` resolves an explicit overlay before the four project source roots, builds only the requested import closure, and records its source and compiler identity. Its output directory should be separate for each proposal. The patch sets are separated so the small import change can be adopted independently of the larger simulation and logical refactors.

The repository's full endpoint build commands remain the appropriate final gate when a combined production checkpoint is selected. The packet's targeted checks are designed to avoid unnecessary repeats of expensive unchanged site checks while establishing that the proposed interfaces elaborate.

B Source guide

All links below point to the pinned commit. The companion measurement JSON files provide the exact paths and hashes used by this report.

1. `KEndpoint215.lean` and `KEndpoint.lean`: literal-machine endpoint and generic composition.
2. `ZFNative.lean`, `NativeTheory254.lean`, `ConstructibleChoice.lean`, and `KQ1ZF.lean`: the semantic import cut.
3. `KSim.lean`: current Boolean coverage, physical representation, and the two-direction halting simulation.
4. `KDecoderPrefix215.lean`, `K215Decoder.lean`, and `DecoderSourceBinding.lean`: exact primitive entry and source/role binding.
5. `KSites215.lean`: current leaf coverage assembled from nine batches.
6. `TarskiDerived.lean`, `KThmK.lean`, and `TarskiCalculus.lean`: the shared named-calculus prototype.
7. Phase-nine summary, Lean endpoint-check instructions, and latest implication-core status: historical and current verification boundaries.
8. Normalized Rocq receipt and current Rocq simulation: distinct source checkpoints used in the comparison.

C Attribution

The underlying compact theory, proof enumerator, register compiler, Turing machine, and existing formal proofs belong to the ProveIt project and the contributors credited in its preserved notices, including Elliot Glazer, Jeff Ketchersid, Andrew James Wade, BusyCoq, and the Coq Library of Undecidability within their respective scopes. This packet refactors and extends that work. It does not claim priority for the existing normalized certificate, condensed-detachment proof, exact decoder prefix, shared Home return, or the earlier proposal to parameterize the named-calculus derivations.

The original source headers are retained in modified files. The supplied attribution notice points to the repository's inherited terms. Repository material is not relicensed merely by inclusion in a research packet.